

WebSockets

Tvorba internetových aplikací

RNDr. Richard Ostertág, PhD.

KI FMFI UK Bratislava

ostertag@dcs.fmph.uniba.sk

Sources

- Mozilla Developer Network: WebSockets
 - <https://developer.mozilla.org/en/docs/WebSockets>
- The WebSocket API
 - <http://www.w3.org/TR/websockets/>
- The WebSocket Protocol
 - <http://tools.ietf.org/html/rfc6455>
- The Definitive Guide to HTML5 WebSocket
 - <http://www.amazon.com/dp/1430247401>

Before WebSockets

- HTTP Polling
 - the client makes at regular intervals a request to the server to see if there is any information
- HTTP Long Polling
 - if the server does not have any information, it holds the request open until it has information for the client, or until it reaches the end of a designated timeout
 - at that point, the client re-requests the information from the server
- HTTP Streaming
 - the server never signals to complete the HTTP response (the connection remains open continuously)
 - proxies and firewalls may buffer the response, resulting in increased latency of the message delivery

Benefits of WebSockets

- Performance
 - saves bandwidth, CPU power, and latency
- Simplicity
 - simple API
- Standards
 - WebSocket API
 - WebSocket Protocol
 - enables to build other standard protocols on top of it
- HTML5
 - part of an effort to provide advanced capabilities to HTML5 applications

WebSocket API – constructor

- create new WebSocket connection
 - var ws = new
WebSocket("ws://echo.websocket.org/echo");
- connecting to the server with multiple protocol choices
 - RFC 6455 use term “subprotocols”
 - var ws = new
WebSocket("ws://echo.websocket.org/echo",
["com.kaazing.echo", "example.protocol"]);

WebSocket Events – Open

- once the server responds to the WebSocket connection request, the open event fires
 - at this moment connection is established
 - no sends before this
 - can check for `ws.readyState == ws.OPEN // 1`
- event handler for the WebSocket connection opening
 - ```
ws.onopen = function(e) {
 console.log("Connection open...");
};
```

# WebSocket Events – Message

- event handler for receiving text messages
  - `ws.onmessage = function(e) {  
    if (typeof e.data === "string") {  
        console.log("String message received", e, e.data);  
    } else {  
        console.log("Other message received", e, e.data);  
    }  
};`

# WebSocket Events – Message

- set binaryType to blob (blob is the default)
  - `ws.binaryType = "blob";`
- event handler for receiving Blob messages
  - `ws.onmessage = function(e) {  
    if(e.data instanceof Blob) {  
        console.log("Blob message received", e.data);  
        var blob = new Blob(e.data);  
    }  
};`



# WebSocket Events – Message

- set binaryType to ArrayBuffer messages
  - `ws.binaryType = "arraybuffer";`
- event handler for receiving ArrayBuffer messages
  - ```
ws.onmessage = function(e) {  
    if(e.data instanceof ArrayBuffer) {  
        console.log("ArrayBuffer Message Received", e.data);  
        // e.data is an ArrayBuffer  
        // create a byte view of that object  
        var a = new Uint8Array(e.data);  
    }  
};
```

WebSocket Events – Error

- event handler for errors in the WebSocket object
 - `ws.onerror = function(e) {
 console.log("WebSocket Error: " , e);
 //Custom function for handling errors
 handleErrors(e);
};`

WebSocket Events – Close

- event handler for closed connections
 - `ws.onclose = function(e) {
 console.log("Connection closed", e);
};`

WebSocket Methods – send

- wait for
 - `ws.readyState == WebSocket.OPEN`
 - or Open event (in onopen handler)
- send a text message
 - `ws.send("Hello WebSocket!");`
- send a Blob
 - `var blob = new Blob(["blob contents"]);`
 - `ws.send(blob);`
- send an ArrayBuffer
 - `var a = new Uint8Array([8,6,7,5,3,0,9]);`
 - `ws.send(a.buffer);`

WebSocket Methods – close

- close the WebSocket connection
 - `ws.close();`
 - `ws.close(code, reason);`
- close the WebSocket connection because the session has ended successfully
 - `ws.close(1000, "Closing normally");`
- code:
 - optional numeric value indicating the status code explaining why the connection is being closed
 - default value of 1000
 - indicating a normal "transaction complete" closure
 - list of status codes:
 - https://developer.mozilla.org/en-US/docs/Web/API/CloseEvent#Status_codes

WebSocket Object Attributes

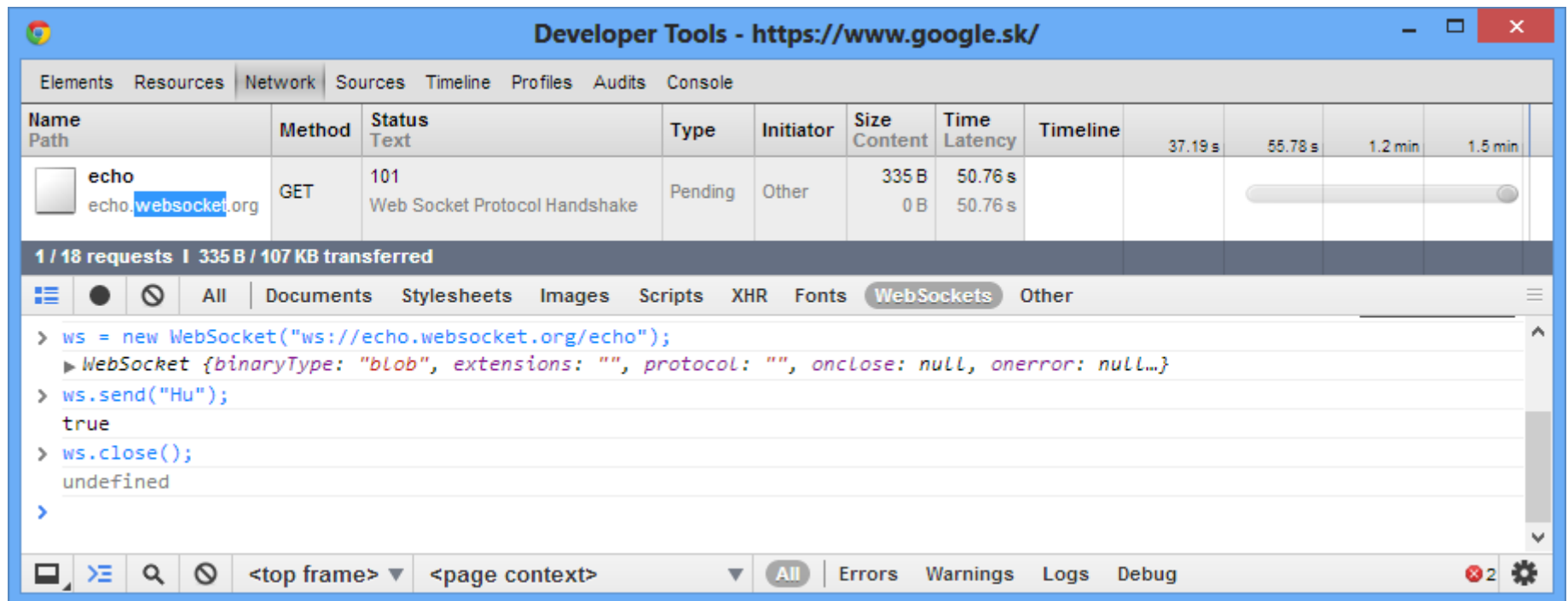
- `readyState`
 - `CONNECTING`: The connection is in progress but has not been established.
 - `OPEN`: The connection has been established. Messages can flow between the client and server.
 - `CLOSING`: The connection is going through the closing handshake.
 - `CLOSED`: The connection has been closed or could not be opened.
- `bufferedAmount`
 - to check the number of bytes that have been queued but not yet transmitted to the server
- `protocol`
 - contains the name of the protocol chosen by the WebSocket server during the opening handshake.

WebSocket Servers

- see list on
 - <http://refcardz.dzone.com/refcardz/html5-websocket>
- Apache ActiveMQ
 - <http://activemq.apache.org/>
- RabbitMQ (Erlang)
 - <http://github.com/videlalvaro/>
 - rabbitmq-websockets
- Socket.io (Node.js)
 - <http://socket.io/>
- Watersprout (PHP)
 - <http://github.com/chrisnetonline/WaterSpout-Server/blob/master/server.php>

Inspecting WebSocket Traffic

- Google Chrome Developer Tools
 - Ctrl+Shift+I (alebo F12), Network, WebSockets



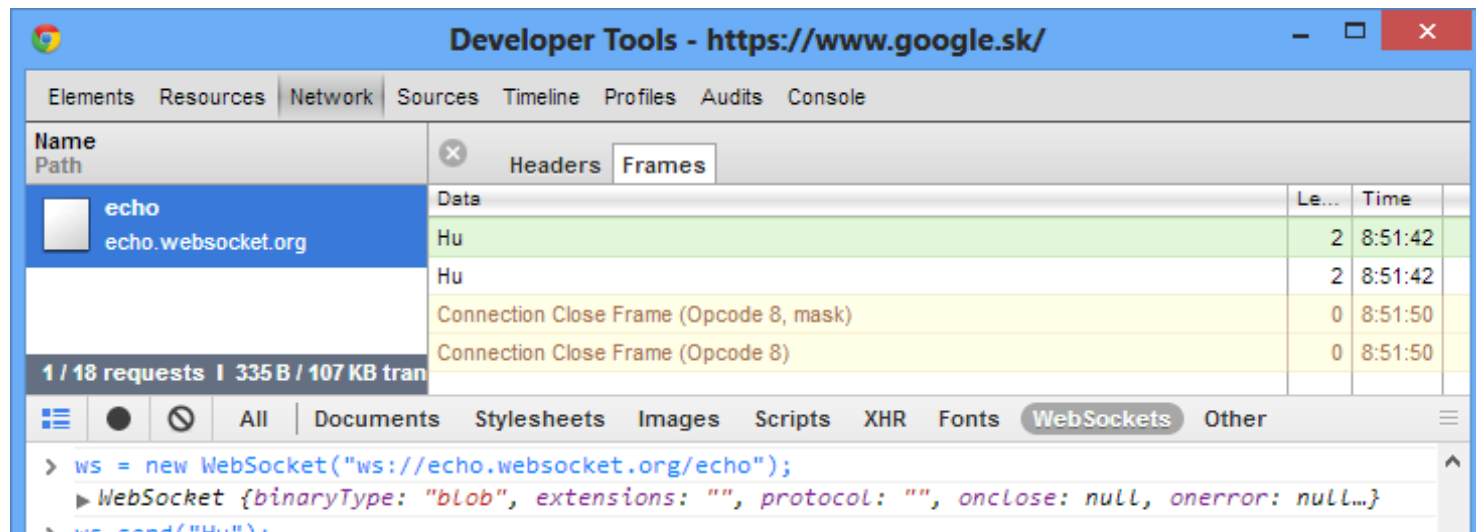
The screenshot shows the Google Chrome Developer Tools interface for the URL `https://www.google.sk/`. The **Network** tab is active, displaying a list of requests. The first request, named `echo` with path `echo.websocket.org`, is a `GET` request with a status of `101` (Web Socket Protocol Handshake). The request is currently in a `Pending` state. The **Size** column shows `335 B` and `0 B`, and the **Time** column shows `50.76 s` and `50.76 s`. Below the request list, a summary bar indicates `1 / 18 requests` and `335 B / 107 KB transferred`. The **WebSockets** sub-tab is selected, showing the JavaScript code for the WebSocket connection:

```
> ws = new WebSocket("ws://echo.websocket.org/echo");
  ► WebSocket {binaryType: "blob", extensions: "", protocol: "", onclose: null, onerror: null...}
> ws.send("Hu");
  true
> ws.close();
  undefined
>
```

The bottom of the interface shows the **Console** tab with filters for `All`, `Errors`, `Warnings`, `Logs`, and `Debug`. The `All` filter is currently selected.

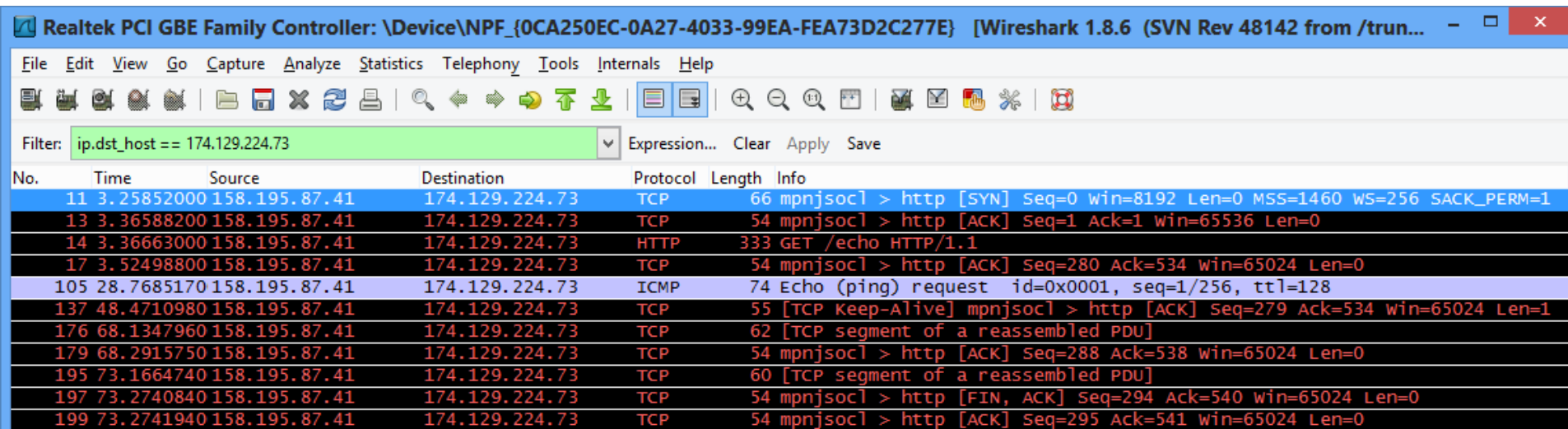
Inspecting WebSocket Traffic

- Google Chrome Developer Tools
 - click on Name to see or refresh list of frames
 - green frames -> send
 - white frames -> reply
 - yellow -> close (send and reply)



Inspecting WebSocket Traffic

- <http://www.tcpdump.org/>
- <http://www.wireshark.org/>
 - for Windows 8 download WinPcap from:
 - <http://www.winpcap.org/install/default.htm>



Realtek PCI GBE Family Controller: \Device\NPF_{0CA250EC-0A27-4033-99EA-FEA73D2C277E} [Wireshark 1.8.6 (SVN Rev 48142 from /trun...]

File Edit View Go Capture Analyze Statistics Telephony Tools Internals Help

Filter: `ip.dst_host == 174.129.224.73` Expression... Clear Apply Save

No.	Time	Source	Destination	Protocol	Length	Info
11	3.25852000	158.195.87.41	174.129.224.73	TCP	66	mpnjsoc1 > http [SYN] Seq=0 win=8192 Len=0 MSS=1460 WS=256 SACK_PERM=1
13	3.36588200	158.195.87.41	174.129.224.73	TCP	54	mpnjsoc1 > http [ACK] Seq=1 Ack=1 win=65536 Len=0
14	3.36663000	158.195.87.41	174.129.224.73	HTTP	333	GET /echo HTTP/1.1
17	3.52498800	158.195.87.41	174.129.224.73	TCP	54	mpnjsoc1 > http [ACK] Seq=280 Ack=534 win=65024 Len=0
105	28.7685170	158.195.87.41	174.129.224.73	ICMP	74	Echo (ping) request id=0x0001, seq=1/256, ttl=128
137	48.4710980	158.195.87.41	174.129.224.73	TCP	55	[TCP Keep-Alive] mpnjsoc1 > http [ACK] Seq=279 Ack=534 win=65024 Len=1
176	68.1347960	158.195.87.41	174.129.224.73	TCP	62	[TCP segment of a reassembled PDU]
179	68.2915750	158.195.87.41	174.129.224.73	TCP	54	mpnjsoc1 > http [ACK] Seq=288 Ack=538 win=65024 Len=0
195	73.1664740	158.195.87.41	174.129.224.73	TCP	60	[TCP segment of a reassembled PDU]
197	73.2740840	158.195.87.41	174.129.224.73	TCP	54	mpnjsoc1 > http [FIN, ACK] Seq=294 Ack=540 win=65024 Len=0
199	73.2741940	158.195.87.41	174.129.224.73	TCP	54	mpnjsoc1 > http [ACK] Seq=295 Ack=541 win=65024 Len=0

Inspecting WebSocket Traffic

- right click on any packet
- select Follow TCP Stream
- use Hex Dump to see unprintable WebSocket communication

